

CHAPTER 08

소프트웨어 프로세스 모델 이해하기

폭포수 · 반복적 · 나선형 · 애자일, 그리고 객체 지향의 표준 Unified Process

Do it! 클린 프로그래밍 · 스터디

목차

AGENDA

8-1

소프트웨어 프로세스 모델이란?

정의 · 필요성 · 역사

8-2

폭포수 모델

선형·순차 진행과 V-모델(V&V)

8-3

반복적 모델

증분형 · 진화형

8-4

나선형 모델

위험 분석 중심의 4단계 반복

8-5

애자일

스프린트 · 스크럼 · XP · 칸반

8-6

Unified Process

객체 지향의 실제 표준

소프트웨어 프로세스 모델이란?

소프트웨어를 어떻게 개발할 것인지 전체 흐름을 체계화한 모형 (개발 생애 주기의 추상적 표현)

레시피에 비유하면

요리에 레시피가 있듯, 개발에도 오랜 경험으로 다져진 '레시피'가 있다. 적절한 모델을 고르면 같은 시행착오를 반복하지 않고 효율적·안정적으로 개발할 수 있다.

모델의 역사 — 2가지로 통합

1960~2000년대

폭포수 · V-모델 · 증분형 · 진화형 · 애자일

2000년대~현재

폭포수 + 반복적 → Unified Process로 통합

왜 필요할까? — 3가지 역할

1

체계적인 개발 절차

각 단계를 명확히 정의 → 일관된 업무 수행, 이해관계자와 원활한 의사소통

2

효율적인 자원 관리

시간·비용을 효과적으로 분배, 산출물 문서화로 유지 보수가 쉬워짐

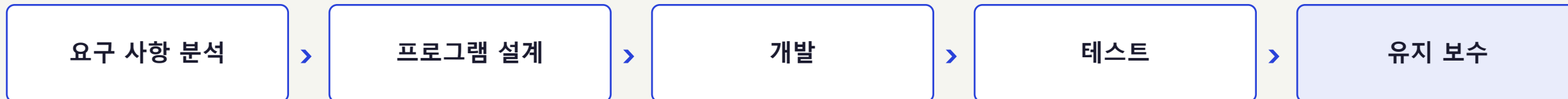
3

높은 수준의 위험 관리

초기부터 위험을 식별·대응 (특히 반복적·점진적 모델이 강점)

폭포수 모델 — 단계별 진행과 문서화

선형적·순차적 접근. 폭포처럼 아래로 흐르며, 이전 단계로 되돌아가기 어렵다



단계별 산출물

단계	산출물	핵심 내용
요구 사항 분석	요구 사항 정의서	기능적·비기능적 요구 사항 기술
프로그램 설계	설계 사양서	아키텍처·DB 설계 문서 작성
개발	소스 코드	프로그래밍 언어로 코드 파일 작성
테스트	테스트 결과서	입력·예상 결과·실제 결과 기록

장점

단계별 명확한 산출물 → 일정 관리가 쉽고 진행 상황 파악이 빠름

단점

요구 사항 변경에 유연하지 못함. 오류·버그가 주로 마지막 테스트 단계에서 발견됨 → 초기 의사소통·테스트 강화로 보완 필요

요구 사항이 명확한 프로젝트라면 지금도 자주 쓰인다.

V-모델 — 테스트 프로세스 강화

폭포수를 확장. 개발 단계와 테스트 단계를 1:1로 대응시켜 V자 형태로 검증

검증 (Verification)

산출물이 요구 사항을 충족하는지 — '제대로 만들고 있는가'

요구 사항 분석 니즈 분석 → 기능·비기능 요구 정리

시스템 설계 필요 기술 파악, 구현 가능성 판단

아키텍처 설계 모듈 인터페이스·관계·의존성, 통합 테스트 계획

단위 설계 기능 로직을 의사 코드 수준으로, 단위 테스트 계획

확인 (Validation)

실제로 요구 사항대로 동작하는지 — '제대로 된 것을 만들었는가'

단위 테스트 화이트박스 · 개별 함수/메서드 로직 검증

통합 테스트 블랙박스 · 모듈 간 인터페이스·상호작용

시스템 테스트 실사용 유사 환경 · 성능 등 시스템 레벨

인수 테스트 신뢰성·배포 준비 완료 여부 최종 확인

핵심: 오류를 조기에 발견할수록 전체 일정과 비용을 크게 단축할 수 있다.

반복적 모델 — 지속적 향상

일부를 개발하고 반복해 개선. 증분형과 진화형으로 나뉜다

증분형 모델

incremental model

전체 시스템을 여러 모듈로 분해, 핵심 모듈부터 점진적으로 개발·릴리스

한 증분 = 하나의 폭포수 사이클 → 폭포수 모델의 변형

- ✓ 초기 요구 사항이 명확할 때 적합, 증분 병렬 개발 가능
- ✓ 신규 도입 충격·후반 통합 부담 완화
- ✗ 여러 증분 관리 부담, 요구 변경 대응이 어려움

진화형 모델

evolutionary model

MVP(핵심 기능만 갖춘 초기 제품)를 먼저 만들고, 피드백을 반영하며 지속 발전

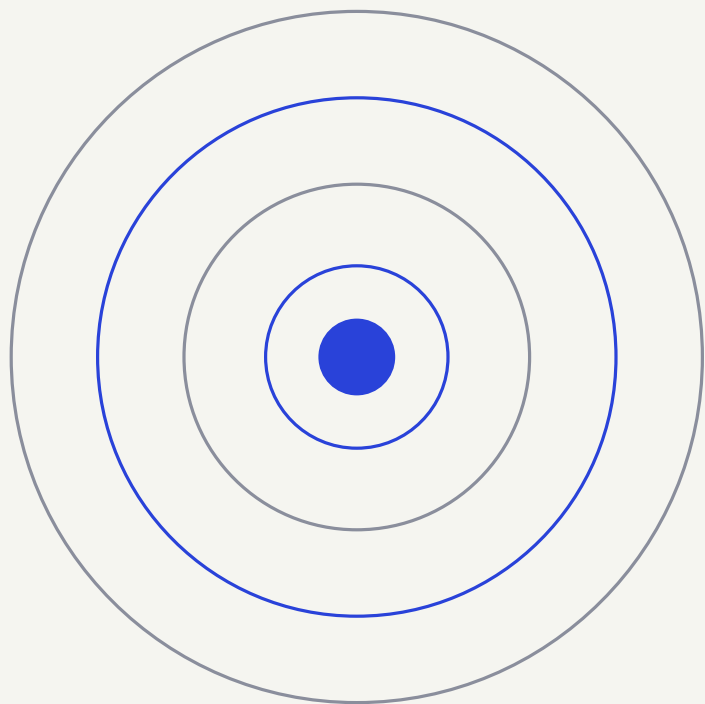
운영 → 피드백 → 개선 사이클 반복

- ✓ 초기 요구 사항이 불명확할 때 적합
- ✓ 완성도를 점차 향상, 유연한 대응
- ✗ 빈번한 릴리스 시 버전 관리·비용·일정 부담

폭포수 vs 반복적 : 변경 대응(낮음↔높음) · 문서 중요도(높음↔프로토타입) · 규모(소규모↔대규모까지)

나선형 모델 — 위험 최소화

위험 분석 단계를 별도로 두어, 점차 완벽한 시스템을 구축한다



이 4단계를 계속 반복하며 위험을 줄이고 시스템을 완성

1

목표 설정

요구 사항 분석·타당성 검토 후, 사이클 목표 수립

2

위험 분석

예상 위험 요소를 식별하고 해결 방안 마련 (조기 최소화)

3

구현 및 테스트

적합한 개발 모델 선택, 단위 → 통합 → 시스템 테스트

4

고객 평가 및 다음 계획

평가로 반복 여부 결정, 인수 테스트로 적합성 최종 검토

애자일 — 신속한 개발 경험

짧은 주기로 릴리스를 반복해, 요구 변화에 유연하고 신속하게 대응

애자일 소프트웨어 개발 선언문

공정과 도구보다 **개인의 상호 작용**을,
포괄적인 문서보다 **동작하는 소프트웨어**를,
계약 협상보다 **고객과의 협력**을,
계획에 따르기보다 **변화에 대응**하기를
가치 있게 여긴다.

핵심 가치

협력 통찰·아이디어 공유로 시너지, 문제를 신속 해결

피드백 내부(개발자 스스로 확인) + 외부(고객·사용자 평가)

애자일 프로세스 구조

제품 백로그

필요한 항목 전체 목록 (지속 진화)

스프린트 백로그

이번 스프린트에 개발할 항목

스프린트 (2~4주)

짧은 반복 개발 주기 + 매일 일일 스크럼

제품 개선 반영

결과물 평가·배포 결정, 개선 반영

애자일 — 개발 방식과 장·한계

스프린트 5단계를 돌며, 다양한 방식으로 구현된다

1. 분석



2. 설계



3. 개발



4. 테스트



5. 검토

애자일을 적용한 대표 방식

스크럼

가장 대표적. 백로그 + 스프린트를 핵심으로, 매 스프린트마다 기능·개선 반영

XP

익스트림 프로그래밍. 반복 주기를 극히 짧게, 빠른 프로토타입·지속 배포

칸반

작업을 카드로 시각화해 흐름 관리. JIRA 등에서 보드 제공

장점

계획 시간 최소화 · 버그 신속 해결 · 피드백 즉각 반영 → 프로토타입을 빠르게 완성하고 시장 출시 시간(Time to Market) 단축

한계

유지 보수 부담 ↑ · 방향성 급변 시 모델 붕괴 위험 · 잦은 협업의 심리적 부담 · 끊임없는 신기술 학습 부담

Unified Process — 객체 지향의 표준

UML과 함께 OMG가 공개한 통합 개발 프로세스. 보통 3주 단위로 반복



★ 상세화 단계가 끝나면 시스템 아키텍처가 확정되는, 가장 핵심적인 단계

4단계 요약

개념화 초기 평가, 시스템 구성 환경 파악

상세화 구조 정의·유스 케이스 도출, 주요 위험 집중 분석

구축 배포 가능한 품질까지 개발·테스트 반복

전이 운영 환경에 배포, 사용자 교육·인수인계

여러 방법론을 조합한 UP

폭포수 개념화: 목표·범위 정의, 초기 요구 수집

나선형 상세화: 아키텍처·요구 위험 조기 발견

프로토타입 주요 기능 정의 후 프로토타입으로 검증

반복적 증분형·진화형으로 점진적 구축

★

한눈에 정리 — 모델 선택 기준

정답은 없다. 형태·규모·기능에 맞게 고르고, 섞어 쓸 수도 있다

모델	이럴 때 적합	주의할 점
폭포수 / V-모델	요구 사항이 명확 · 문서·단계가 중요	변경에 약함
증분형	요구가 명확하고 모듈로 쪼갤 수 있을 때	여러 증분 관리 부담
진화형	요구가 불명확 · MVP로 빠르게 검증	버전 관리 부담
나선형	위험이 크고 비싼 프로젝트	관리 복잡도 ↑
애자일	변화가 잦고 빠른 출시가 중요	협업·학습 부담
UP	객체 지향 · 중대형 프로젝트 표준	프로세스 학습 곡선

정리하며

모델은 도구다. 상황에 맞게 고르고, 섞어 쓰자.

폭포수의 명확함, 반복적·나선형의 위험 관리, 애자일·UP의 유연함 — 핵심 가치를 알아 두면 협업할 때 든든하다.

질문 환영합니다 💬